

PTfS23: Assignment 0: Warming up

Ashwin Varkey (23115420)

Benchmark code: $\int_0^1 \frac{4}{1+x^2} dx.$

In this implementation, we use $\text{delta_x} = 1.0 / \text{SLICES}$ to compute the width of each rectangle, and we sum up the areas of the rectangles to approximate the integral. The constant SLICES is used to convert the wall time to CPU cycles.

```
#include <iostream>
#include "timing.h"
double freq=2e9
int main() {
    int SLICES = 1000000000;
    double sum = 0.0, delta_x = 1.0 / SLICES;
    double wcTime, wcTimeStart, wcTimeEnd;
    wcTimeStart = getTimeStamp();
    for (int i = 0; i < SLICES; i++) {
        double x = (i + 0.5) * delta_x;
        sum += 4.0 / (1.0 + x * x);
    }
    double Pi = sum * delta_x;
    wcTimeEnd = getTimeStamp();
    wcTime = wcTimeEnd - wcTimeStart;
    std::cout << "Approximate value of Pi: " << Pi << std::endl;
    std::cout << "Time taken (wall time): " << wcTime << " seconds" << std::endl;
    double cycles_per_iter = (freq * wcTime) / SLICES;
    std::cout << "CPU cycles per iteration: " << cycles_per_iter << std::endl;
    return 0;
}
```

1. How many CPU cycles does the code take to execute per iteration of the loop? Describe how you arrived at this number from your measurements

```
icpx -o3 -xHost -o double Final.cpp timing.c timing.h | module load intel (intel/2022.1.0)
```

Model name: Intel(R) Xeon(R) Platinum 8360Y CPU @ 2.40GHz

L1d cache: 48K L1i cache: 32K L2 cache: 1280K L3 cache: 55296K

CPU max MHz: 3500.0000 CPU min MHz: 800.0000

```
srunk --cpu-freq=2000000-2000000 ./double>Results.txt
```

Output:

Approximate value of Pi: 3.14159

Time taken (wall time): 1.00536 seconds

CPU cycles per iteration: 2.01072

Total CPU cycles = wall time * clock rate = 1.005 s * 2 GHz = 2.01 * 10⁹ cycles

CPU cycles per iteration = Total CPU cycles / number of iterations (SLICES) = 2.01 * 10⁹ cycles / 1000000000 iterations = 2.01 cycles/iteration

2. What is the performance of the code? Is Flops/s a good performance metric here? Discuss different options for appropriate performance metrics in this code.

- CPU cycles per iteration or wall-clock time can be appropriate performance metrics for this code.
- The reported value of Pi is close to the true value, indicating that the code has a reasonable level of accuracy. The wall-clock time for this code is around 1.005 seconds, which can be used as a performance metric to measure how long the code takes to execute. Additionally, CPU cycles per iteration is around 2 cycles, which indicates the efficiency of the code in terms of CPU usage.
- Flops/s is not a good performance metric here because the code only performs simple arithmetic operations (addition, multiplication, division) and does not use any special hardware instructions or parallelization. Instead, a better metric would be the time or cycles per iteration (MIPS), which reflects the efficiency of the code in performing the required computations.

Wall-clock time: This measures the actual time taken to execute the entire program from start to finish, including any delays running on the system. It is a good metric to measure the overall performance of the program in terms of how long it takes to complete a task.

CPU cycles per iteration of the loop: This measures the number of CPU cycles required to execute each iteration of the loop. It gives an idea of how efficient the program is in terms of using the CPU resources. This metric is useful for comparing different implementations of the same algorithm to see which one is more efficient.

Flops/s: This measures the number of floating-point operations that can be performed per second. It is a good metric for measuring the performance of programs that involve a lot of floating-point operations, such as scientific simulations or machine learning algorithms.

MIPS: Millions of Instructions per Second is a metric that measures the number of instructions that can be executed per second. It is a useful metric for measuring the performance of programs that involve a lot of instructions, such as compilers or interpreters.

In summary, the appropriate performance metric depends on the type of program being run and the specific goals of the performance evaluation. Wall-clock time and CPU cycles per iteration are good metrics for measuring the efficiency of code, while Flops/s is a good metric for measuring the performance of programs that involve a lot of floating-point operations. MIPS is useful for measuring the performance of programs that involve a lot of instructions.

3. Discuss what happens (in terms of performance and accuracy) if you change the code to use single-precision floating-point numbers! Are there any relevant code changes beyond data types? Why do you not get a reasonable accuracy for π ?

Change in code switch double datatype to float in code and no relevant changes beyond datatypes

```
#include <iostream>
#include "timing.h"
double freq=2e9
int main() {
    int SLICES = 1000000000;
    float sum = 0.0f, delta_x = 1.0f / SLICES;
    float wcTime, wcTimeStart, wcTimeEnd;
    wcTimeStart = getTimeStamp();
    for (int i = 0; i < SLICES; i++) {
        float x = (i + 0.5f) * delta_x;
        sum += 4.0f / (1.0f + x * x);
    }
    float Pi = sum * delta_x;
    wcTimeEnd = getTimeStamp();
    wcTime = wcTimeEnd - wcTimeStart;
    std::cout << "Approximate value of Pi: " << Pi << std::endl;
    std::cout << "Time taken: " << wcTime << " seconds" << std::endl;
    float cycles_per_iter = (freq * wcTime) / SLICES;
    std::cout << "CPU cycles per iteration: " << cycles_per_iter << std::endl;
    return 0;
}
```

Output:

Approximate value of Pi: 0.0671089

Time taken: 7 seconds

CPU cycles per iteration: 14

The Pi value is not accurate, with a value of approximately 0.066 and wall time is much higher than double precision case. This is due to the accumulation of rounding errors in the summation process caused by the lower precision of single-precision floating-point numbers compared to double-precision floating-point numbers. While higher CPU cycles per iteration (14) in single precision case compared to double might indicate better performance, it's important to consider other factors as well when evaluating CPU performance. Hence it is important to ensure performance gain with reasonable accuracy of pi while evaluating.

4. Run the (double) code, but this time do not fix the clock frequency. Calculate the actual frequency

Wall clock time is calculated using start and end times. The number of CPU cycles used by the loop is measured by using the `__rdtsc()` intrinsic function in Intel compiler that reads the value of the Time Stamp Counter (TSC) register on modern x86 processors that counts the number of clock cycles since the system was last reset.

To measure the number of CPU cycles used by the loop, `tscStart` and `tscEnd = __rdtsc();` are used to read the value of the TSC register at the start and end of the loop, respectively. The difference between the TSC values at the start and end of the loop gives us the number of CPU cycles used by the loop.

Total CPU cycles = wall time * clock rate (Frequency) | Actual Frequency = total CPU cycles/Wall time

The actual CPU frequency can vary depending on factors such as CPU load, temperature, and power. It is crucial to measure the actual frequency during the execution, rather than assuming a fixed value.

```
#include <iostream>
#include "timing.h"
#include <stdio.h>
#include <stdlib.h>
int main() {
    int SLICES = 1000000000;
    double sum = 0.0, delta_x = 1.0 / SLICES;
    unsigned long long tscStart, tscEnd, tscElapsed;
    double wcTime, wcTimeStart, wcTimeEnd;
    wcTimeStart = getTimeStamp();
    tscStart = __rdtsc();
    for (int i = 0; i < SLICES; i++) {
        double x = (i + 0.5) * delta_x;
        sum += 4.0 / (1.0 + x * x);
    }
    double Pi = sum * delta_x;
    tscEnd = __rdtsc();
    wcTimeEnd = getTimeStamp();
    wcTime = wcTimeEnd - wcTimeStart;
    tscElapsed = tscEnd - tscStart;
    double actual_frequency = (tscElapsed / wcTime);
    double cycles_per_iter = (actual_frequency * wcTime) / SLICES;
    std::cout << "Approximate value of Pi: " << Pi << std::endl;
    std::cout << "Time taken: " << wcTime << " seconds" << std::endl;
    std::cout << "CPU cycles per iteration: " << cycles_per_iter << std::endl;
    std::cout << "Actual CPU frequency in GHz:" << actual_frequency / 1e9;
    return 0;
}
```

Output:

Approximate value of Pi: 3.14159

Time taken: 0.853216 seconds

CPU cycles per iteration: 2.04

Actual CPU frequency in GHz: 2.39434